# Session 6: Python Recap Lab

DIGITAL AND PUBLIC POLICY STREAM

Python Tutorial
Practice for Data Analysis

# Session outline

## Warm-ups

Numbers, text and lists
Methods and stacks

## Combine ideas

Tuples, sets and dicts
Loops and functions

## Mini-analysis

Small records to summaries
Plot-ready labels and values

**Online rhythm: predict in notebook, then discuss the next slide**

# Practice 1: Numbers and lists

In notebook (45 seconds): predict both printed lines.

What does index 1 select?

Where does text join the result?

**Python 3**

```
prices = [12, 8, 15]
label = "Total"
print(label + ":", sum(prices))
print(prices[1] * 2)
```

**In notebook**

Post both lines before running the code.

# Practice 1: Discuss the result

The list holds three numbers.

Index 1 selects 8 because indexing starts at 0.

sum(prices) adds the three values.

**Python 3**

```
prices = [12, 8, 15]
label = "Total"
print(label + ":", sum(prices))
print(prices[1] * 2)
```

**Output**

```
Total: 35
16
```

# Practice 2: A list as a stack

In notebook (45 seconds): what are done and tasks?

Which item leaves first?

What do append and pop change?

**Python 3**

```
tasks = ["read", "clean"]
tasks.append("plot")
done = tasks.pop()
print(done)
print(tasks)
```

**In notebook**

Post done and the remaining list.

# Practice 2: Discuss the stack

append adds an item at the end.

pop removes and returns the last item.

This is last in, first out.

**Python 3**

```
tasks = ["read", "clean"]
tasks.append("plot")
done = tasks.pop()
print(done)
print(tasks)
```

**Output**

```
plot
["read", "clean"]
```

# Practice 3: Tuples and sets

In notebook (60 seconds): predict the two lines.

What is one row?

Why do we use a set for regions?

**Python 3**

```
rows = [("N", 2), ("S", 3),
        ("N", 4)]
regions = set()
for region, value in rows:
    regions.add(region)
print(sorted(regions))
print(len(rows))
```

**In chat**

```
Post the unique regions and row
count.
```

# Practice 3: Discuss the data

Each row is a two-value tuple.

The list keeps all three observations in order.

The set removes the repeated region; sorted makes the display stable.

**Python 3**

```
rows = [("N", 2), ("S", 3),
        ("N", 4)]
regions = set()
for region, value in rows:
    regions.add(region)
print(sorted(regions))
print(len(rows))
```

**Output**

```
['N', 'S']
3
```

# Practice 4: Trace the loop

In notebook (60 seconds): list the printed numbers.

Which value does continue skip?

Does the loop else run after break?

**Python 3**

```
for n in range(1, 7):
    if n == 3:
        continue
    if n == 5:
        break
    print(n)
else:
    print("complete")
```

**In notebook**

Post the output and explain else.

# Practice 4: Discuss the loop

Output: 1, 2, 4.

range(1, 7) tries 1 through 6.

continue skips printing 3.

break stops at 5, so loop else does not run.

**Python 3**

```
for n in range(1, 7):
    if n == 3:
        continue
    if n == 5:
        break
    print(n)
```

# Practice 5: Function calls

In notebook (90 seconds): which calls work?

Which parameter has a default?

Explain the roles of / and *.

**Python 3**

```
def scale(x, /, factor=2, *,
          digits=1):
    y = x * factor
    return round(y, digits)

scale(2.7)
scale(x=2.7)
scale(2.7, digits=0)
scale(2.7, 3, 1)
```

**In notebook**

Post valid calls and one reason.

# Practice 5: Discuss the calls

x takes position only.

factor can use position or keyword and defaults to 2.

digits needs a keyword and defaults to 1.

**Python 3**

```
def scale(x, /, factor=2, *,
          digits=1):
    y = x * factor
    return round(y, digits)

print(scale(2.7))
print(scale(2.7, digits=0))
```

**Output**

```
5.4
5.0
```

# Practice 6: Choose a container

## Ordered work

```
from collections import deque
rows = [("N", 2), ("S", 3)]
jobs = deque(["load", "clean"])
```

Chat: what type is one row?
How do we remove the first job?

## Grouping and uniqueness

```
seen = {"N", "S", "N"}
totals = {"N": 2, "S": 3}
```

Chat: which removes repeats?
Which maps a key to a value?

# Practice 6: Discuss the choice

A tuple holds one fixed row; the list keeps row order.

A deque serves a first-in, first-out queue.

A set keeps unique values; a dict maps regions to totals.

**Python 3**

```
from collections import deque
jobs = deque(["load", "clean"])
print(jobs.popleft())
print(list(jobs))
print(sorted({"N", "S", "N"}))
```

**Output**

```
load
['clean']
['N', 'S']
```

# Practice 7: Plan a summary

## Input rows

Three observations:
N has 12, S has 8, N has 10.
Each record is a tuple.

Chat: what is one row?

## Target summary

Keep values of at least 10.
Add retained values by region in a dictionary.

Chat: which region remains, and what is its total?

# Practice 7: Discuss the summary

The if excludes the value 8.

The dictionary adds values by region.

The keys and values can become labels and heights for a chart.

**Python 3**

```
rows = [("N", 12), ("S", 8),
        ("N", 10)]
totals = {}
for region, value in rows:
    if value < 10:
        continue
    old = totals.get(region, 0)
    totals[region] = old + value
print(totals)
```

**Output**

```
{'N': 22}
```

# Hands-on: Mini Analysis

# Hands-on: Summarize five sales

**Use these records**

```
data=[('N',9),('S',5),('N',0),('E',7),('S',4)]
```

write tally(rows, /, *, floor=1).

1. Skip values below floor with continue.
2. Add retained values by region in a dict.
3. Return and print the summary.
4. Name the labels and values for a bar chart.

# Hands-on discussion and solution

Use data from the previous slide.

Why use a dictionary?
Each region gets one total.

Why skip zero?
It is below the default floor of 1.

**Python 3**

```
def tally(rows, /, *, floor=1):
    out = {}
    for key, n in rows:
        if n < floor:
            continue
        old = out.get(key, 0)
        out[key] = old + n
    return out
print(tally(data))
```

**Output**

```
{'N': 9, 'S': 9, 'E': 7}
```

# Recap and readings

Numbers and strings build values; lists hold sequences.
if and loops filter observations.
Functions package a repeatable calculation.
Tuples hold rows; sets remove repeats.
Dictionaries hold grouped summaries.
Next: DataFrame rows and bar-chart labels and values.

---

Python Tutorial: Introduction, Control Flow, Data Structures
Readings from Sessions 1–5